School of Engineering

SFBU

CS570: Big Data Processing & Analytics

Spring 2026

Dr. Ragnar Lesch

Adjunct Professor

What Is "Big Data"?

- Data that doesn’t fit the assumption of a single machine
- Data that arrives continuously or unpredictably
- Data with evolving formats

Big Data is not defined by Size alone, but by the failure of traditional assumptions.

Properties of Modern Data

The 3 V’s:

- Volume: GB -> TB -> PB
- Velocity: Always-on streams
- Variety: Logs, JSON, evolving schemas

Why Petabyte Scale Matters

Early 2000s

- Petabyte-scale data became common at large web companies
- Data no longer fits in RAM or on one disk
- Scanning data becomes the main cost

Real Data Is Messy

Web Era

- Logs and JSON lack fixed schemas
- Fields may be missing or nested
- Schemas evolve over time

Real-World Example: User Event Log

```
{
  // Event 1: Complete record
 {
  "user_id": "abc123",
  "timestamp": "2024-01-15T14:30:00Z",
  "event": "purchase",
  "location": {"city": "SF", "country": "US"},
  "amount": 49.99
 }

 // Event 2: Missing fields, different format
 {
  "user_id": 456,  // ← now a number!
  "timestamp": "Jan 15, 2024 2:31 PM",  // ← format changed
  "event": "click"
  // ← location and amount missing!
 }
}
```

Why Did Data Become Messy?

The Shift from Controlled Systems to Open Ecosystems

1990s: Neat & Controlled

- **Single Organization Control**
  One company owned and designed the entire database
- **Internal Systems Only**
  Data stayed within corporate walls
- **Professional Data Entry**
  Trained staff entered data via validated forms
- **Slow Change Cycles**
  Schema changes required months of planning
- **Limited Data Sources**
  Transactional systems: orders, inventory, payroll

2000s-Today: Messy & Open

- **Multiple Systems Integration**
  Data from hundreds of APIs, microservices, partners
- **User-Generated Content**
  Billions of users creating unstructured data
- **Speed Over Perfection**
  Move fast and break things—ship features quickly
- **Continuous Evolution**
  APIs change weekly, A/B tests add fields randomly
- **Heterogeneous Sources**
  IoT sensors, mobile apps, web logs, social media

Vertical Scaling

1990s–Early 2000s

- Scaling up a single machine
- Add more CPU, RAM, or faster disks to one machine
- Same program, same architecture

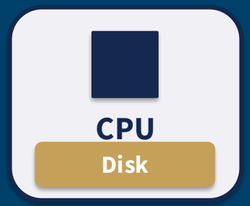
Horizontal Scaling

Early–Mid 2000s: Scaling out across machines

- Emerged prominently in the early to mid-2000s
- Add more machines instead of making one machine bigger
- Data and computation distributed across nodes
- Parallelism and fault tolerance by design

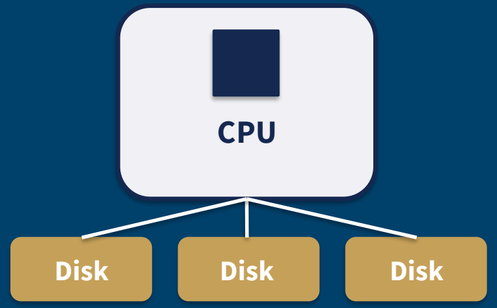
Scaling Paradigms

Single Node



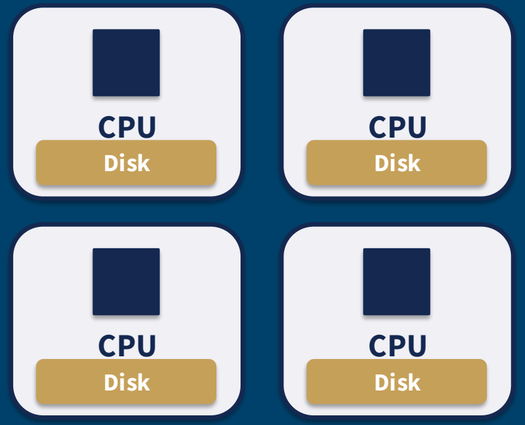
Centralized Storage & Compute

Single Node with Network Storage



The Network Bottleneck

Shared Nothing



Distributed Storage & Compute

Move Compute To The Data

2000s

- Network is the slowest resource
- Moving data is expensive
- Computation should run where data lives

Big Data Systems Timeline

From Research to Production

2003/4

Google publishes GFS & MapReduce papers

2006

Hadoop created at Yahoo!

2010

Spark released at UC Berkeley

2014

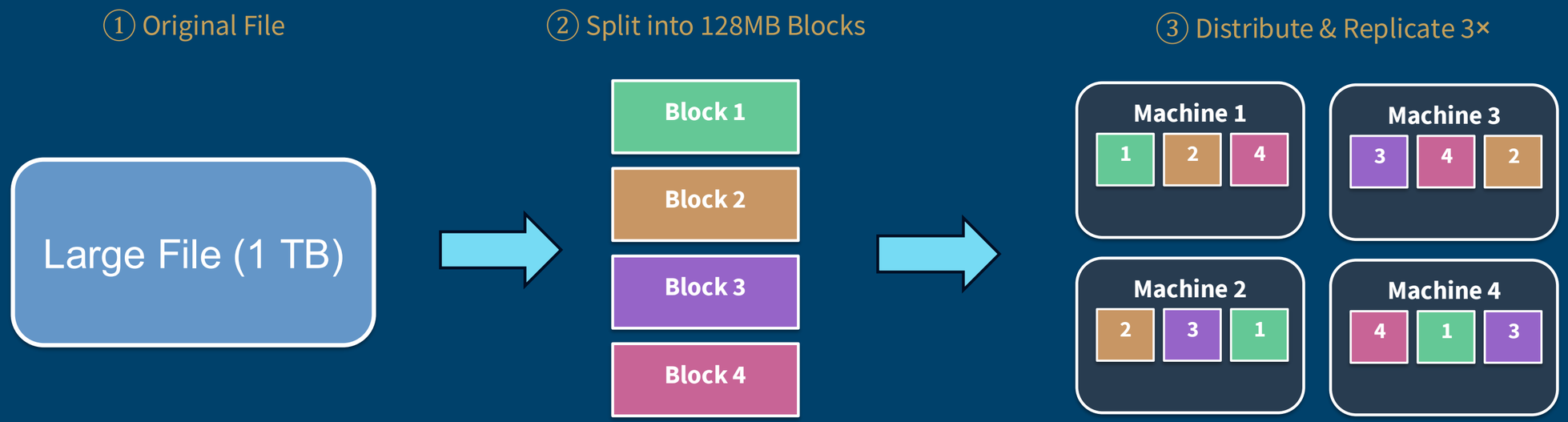
Spark becomes Apache top-level project

2020s

Cloud-native data platforms

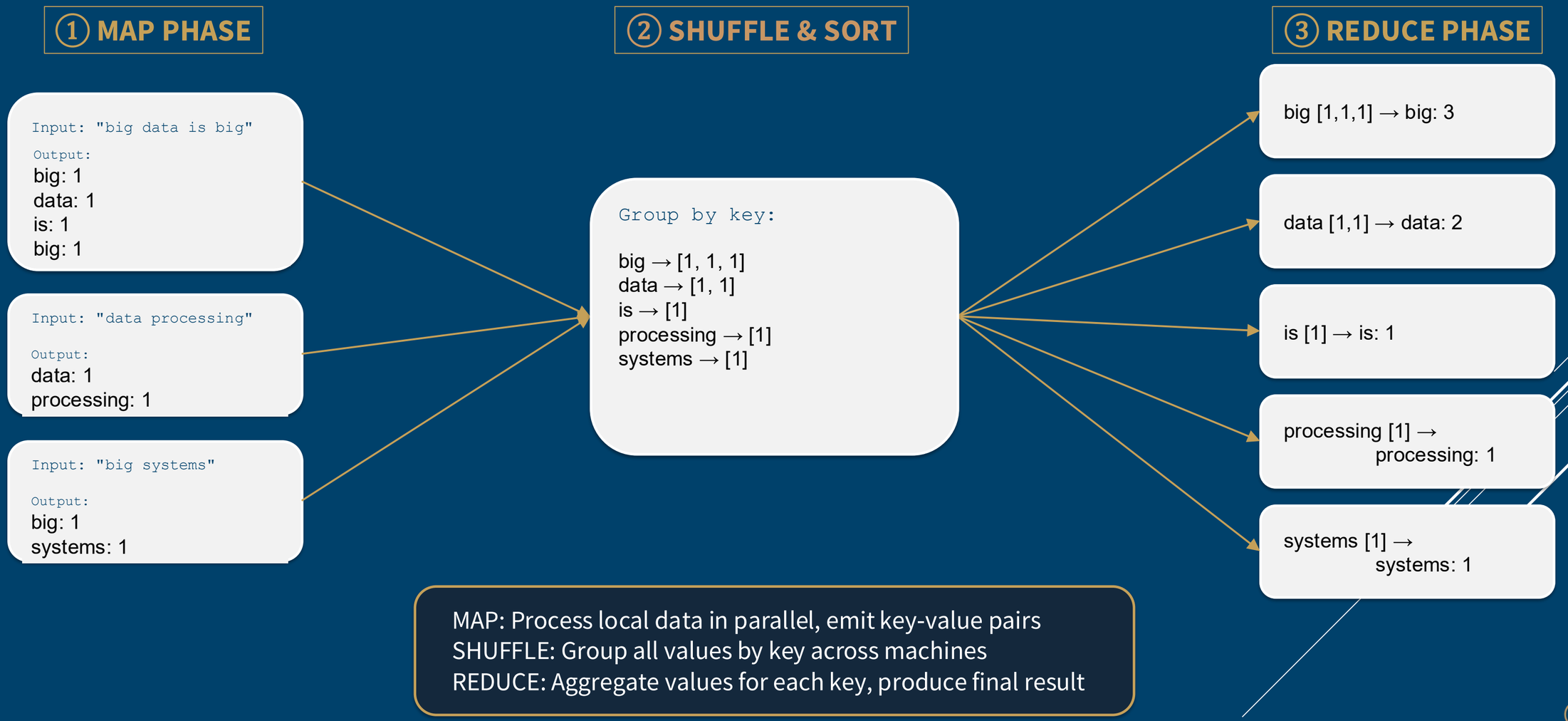
How Hadoop Works

Split, Distribute, Replicate



Each colored block appears on 3 different machines. Machine 1 fails? Blocks 1, 2, 4 still exist on other machines. No data loss.

How MapReduce Works



MAP: Process local data in parallel, emit key-value pairs
SHUFFLE: Group all values by key across machines
REDUCE: Aggregate values for each key, produce final result

Summary for HDFS and MapReduce

Mid-2000s: Two systems working together

1 HDFS: Distributed Storage

- Split & Distribute:
  Large files split into 128MB blocks distributed across cluster
- Replicate for Safety:
  Each block replicated 3 times on different machines
- Data Locality:
  Know which machine has which blocks, send computation to data

2 MapReduce: Distributed Processing

- Map Phase:
  Send code to machines with data, each processing those in parallel
- Shuffle Phase:
  Redistribute intermediate results by key, group related data together
- Reduce Phase:
  Combine results from all machines, produce final output

The Key Principle: Data Locality

HDFS stores data locally on each machine. MapReduce processes that data locally. Only intermediate results move over network.

What Hadoop Was Designed For

Mid-2000s: Strengths of disk-based batch processing

The ideal Hadoop workload:

- Process terabytes or petabytes of data
- Batch job that runs overnight or over the weekend
- Scan through data once, maybe twice
- Doesn't need sub-second response times
- Runs on commodity hardware that fails frequently

Examples of perfect Hadoop use cases:

- Web indexing: crawl the web, build search index, run overnight
- Log analysis: collect all server logs, analyze for patterns, generate daily reports
- Data warehousing: ETL jobs that transform raw data into analytics tables
- Recommendation systems: process all user behavior, compute recommendations weekly

This is what made Hadoop revolutionary in 2006.
Companies could finally process web-scale data without buying million-dollar supercomputers. Yahoo!, Facebook, LinkedIn — they all ran Hadoop on cheap servers and achieved Google-scale processing.

Where Hadoop Starts To Struggle

Late 2000s: Mismatch between design and new workloads

- Repeated passes over the same data
- Interactive or exploratory analysis
- Iterative algorithms such as machine learning
- Slow feedback cycles due to disk-heavy execution

How Spark Solves These Problems

2010s: Spark answers to Hadoop's limitations

- In-Memory Caching
- Lazy Evaluation
- Data Locality Preserved
- Fault Tolerance Without Disk

Result: In practice, many iterative workloads see 10-100x speedups compared to Hadoop.

Spark’s Lazy Evaluation

Build, optimize, then execute

- Build a logical plan first
- Optimize before execution
- Execute only when required

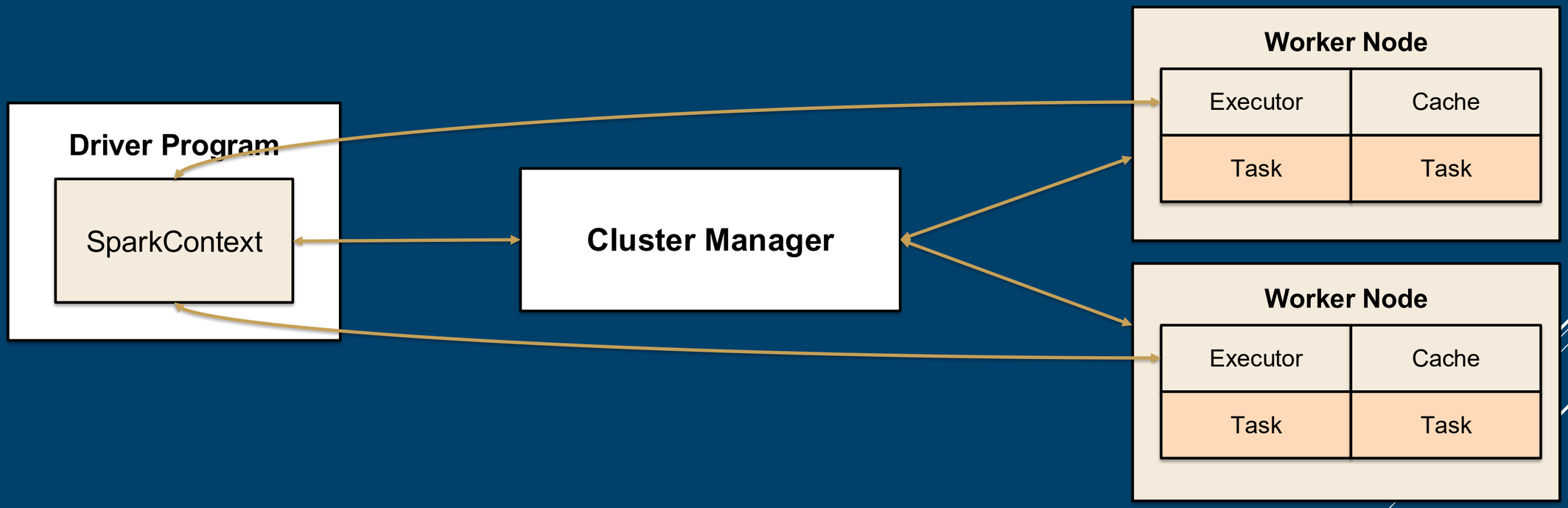
Spark’s Lazy Evaluation

Build, optimize, then execute

```
data = spark.read.file(...)       # Lazy - just planning

filtered = data.filter(...)       # Lazy - just planning

grouped = filtered.groupBy(...)   # Lazy - just planning

result = grouped.count()          # Action! Now Spark executes everything
```

Spark Architecture

2010s–Today



Plans and coordinates work

Allocates resources

Execute tasks and cache data